Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

# Лабораторная работа №2 по курсу

# «Операционные системы»

Группа: М8О-213БВ-26

Студент: Ипала Садат

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 25.10.26

Москва, 2026

# Постановка задачи

**Вариант 9.**

**Рассчитать детерминант матрицы (используя определение детерминанта).**

**Реализовать последовательную и параллельную версии алгоритма.**

**Ограничение максимального количества потоков, работающих в один момент времени, задаётся ключом запуска программы. Провести исследование зависимости ускорения и эффективности алгоритма от входных данных и количества потоков.**

# Общий метод и алгоритм решения

Использованные системные вызовы:

- pthread_create(pthread_t *thread, const pthread_attr_t *attr, void *(*start_routine)(void *), void *arg) — создаёт новый поток.
- pthread_join(pthread_t thread, void **retval) — ожидает завершения потока.
- pthread_t — тип идентификатора потока.
- clock_gettime(clockid_t clk_id, struct timespec *tp) — возвращает текущее время с высокой точностью (используется для замера времени выполнения).
- malloc(size_t size) / free(void *ptr) — выделение и освобождение динамической памяти (для матрицы и массивов аргументов потоков).
- rand() / srand(unsigned int seed) — генерация псевдослучайных чисел для заполнения матрицы.
- strcmp, atoi, fprintf, printf — стандартные функции для парсинга аргументов и вывода результатов.

**Описание работы программы:**

**Математическая основа.**
Определитель матрицы A размера n×n вычисляется по формуле Лейбница:
$\det(A) = \Sigma_{\sigma \in S_n} \operatorname{sgn}(\sigma) \cdot \Pi_{i=1}^{n} A_{i, \sigma(i)}$
где $S_n$ - множество всех перестановок чисел {1, 2, …, n},
$\operatorname{sgn}(\sigma)$ - знак перестановки (+1 для чётной, -1 для нечётной),
$A_{i, \sigma(i)}$ - элемент матрицы в строке i и столбце $\sigma(i)$.
**Сложность алгоритма.**
Количество перестановок равно n!, поэтому алгоритм имеет факториальную сложность O(n!). Это накладывает ограничение на размер матрицы: для n = 10 количество перестановок равно 3 628 800, для n = 11 - уже 39 916 800.
**Последовательная версия.**

Программа перебирает все перестановки от 0 до n!-1. Для каждой перестановки восстанавливается её ранг (алгоритм факториального представления), вычисляется произведение элементов матрицы и знак перестановки. Все произведения суммируются.

**Параллельная версия.**

Диапазон [0, n!) разбивается на T примерно равных блоков, где T - количество потоков, задаваемое ключом запуска. Каждый поток обрабатывает свой блок независимо и сохраняет частичную сумму в собственную структуру. После завершения всех потоков (pthread_join) частичные суммы складываются в главном потоке. Благодаря тому, что каждый поток пишет в свою ячейку памяти, mutex не требуется.

**Ограничение количества потоков.**

Если T > n!, то реально создаётся только n! потоков (не больше, чем количество задач). Значение T передаётся через ключ -t.

**Метрики производительности.**

Ускорение (speedup) вычисляется как:

$S(T) = T_1 / T_T$

где $T_1$ - время выполнения при 1 потоке, $T_T$ - время выполнения при T потоках.

Эффективность (efficiency) вычисляется как:

$E(T) = S(T) / T = T_1 / (T \cdot T_T)$

Идеальные значения: $S(T) = T$ и $E(T) = 1$. На практике $E(T) < 1$ из-за накладных расходов на создание/завершение потоков, синхронизацию и неравномерную загрузку ядер.

**Замер времени.**

Используется clock_gettime(CLOCK_MONOTONIC, …), который даёт монотонное время, устойчивое к изменению системных часов. Перед основным бенчмарком выполняется «прогрев» (warm-up), чтобы стабилизировать состояние кэша процессора.

# Код программы

**main.c**

```
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>
#include <pthread.h>
#include <math.h>
#include <unistd.h>
#include <stdint.h>

typedef struct {
    int **matrix;
    int n;
    uint64_t start_perm;
    uint64_t end_perm;
    double partial_result;
```

```
} worker_args_t;

static uint64_t factorial(int n) {
    uint64_t r = 1;
    for (int i = 2; i <= n; i++) {
        r *= (uint64_t)i;
        if (r == 0) return 0;
    }
    return r;
}

static int **alloc_matrix(int n) {
    int **m = (int **)malloc((size_t)n * sizeof(int *));
    if (!m) return NULL;
    for (int i = 0; i < n; i++) {
        m[i] = (int *)malloc((size_t)n * sizeof(int));
        if (!m[i]) {
            for (int j = 0; j < i; j++) free(m[j]);
            free(m);
            return NULL;
        }
    }
    return m;
}

static void free_matrix(int **m, int n) {
    if (!m) return;
    for (int i = 0; i < n; i++) free(m[i]);
    free(m);
}

static void fill_matrix(int **m, int n, unsigned int seed) {
    srand(seed);
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            m[i][j] = (rand() % 19) - 9;
}

static void print_matrix(int **m, int n) {
    printf("Matrix %dx%d:\n", n, n);
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++)
            printf("%4d ", m[i][j]);
        printf("\n");
    }
    printf("\n");
}

static void permutation_rank(uint64_t k, int n, int *perm) {
```

```
    int used[16] = {0};
    uint64_t fact[16];
    fact[0] = 1;
    for (int i = 1; i <= n; i++) fact[i] = fact[i-1] * (uint64_t)i;

    for (int i = 0; i < n; i++) {
        uint64_t block = fact[n - 1 - i];
        uint64_t idx = (block > 0) ? (k / block) : 0;
        k = (block > 0) ? (k % block) : 0;

        int count = 0;
        int picked = -1;
        for (int v = 0; v < n; v++) {
            if (!used[v]) {
                if ((uint64_t)count == idx) { picked = v; break; }
                count++;
            }
        }
        perm[i] = picked;
        used[picked] = 1;
    }
}

static int permutation_sign(const int *perm, int n) {
    int inv = 0;
    for (int i = 0; i < n; i++)
        for (int j = i + 1; j < n; j++)
            if (perm[i] > perm[j]) inv++;
    return (inv % 2 == 0) ? 1 : -1;
}

static double det_range(int **m, int n, uint64_t start, uint64_t end) {
    double sum = 0.0;
    int *perm = (int *)malloc((size_t)n * sizeof(int));
    if (!perm) return 0.0;

    for (uint64_t k = start; k < end; k++) {
        permutation_rank(k, n, perm);
        double prod = 1.0;
        for (int i = 0; i < n; i++) prod *= (double)m[i][perm[i]];
        sum += (double)permutation_sign(perm, n) * prod;
    }
    free(perm);
    return sum;
}

static void *worker(void *arg) {
    worker_args_t *wa = (worker_args_t *)arg;
    wa->partial_result = det_range(wa->matrix, wa->n, wa->start_perm, wa->end_perm);
```

```
    return NULL;
}

static double det_parallel(int **m, int n, int T) {
    uint64_t total = factorial(n);
    if (total == 0) return 0.0;

    if ((uint64_t)T > total) T = (int)total;
    if (T <= 0) T = 1;

    pthread_t *threads = (pthread_t *)malloc((size_t)T * sizeof(pthread_t));
    worker_args_t *args = (worker_args_t *)malloc((size_t)T * sizeof(worker_args_t));
    if (!threads || !args) {
        free(threads);
        free(args);
        return 0.0;
    }

    uint64_t base = total / (uint64_t)T;
    uint64_t rem = total % (uint64_t)T;
    uint64_t cursor = 0;

    for (int i = 0; i < T; i++) {
        uint64_t chunk = base + ((uint64_t)i < rem ? 1 : 0);
        args[i].matrix = m;
        args[i].n = n;
        args[i].start_perm = cursor;
        args[i].end_perm = cursor + chunk;
        args[i].partial_result = 0.0;
        cursor += chunk;
        pthread_create(&threads[i], NULL, worker, &args[i]);
    }

    double total_result = 0.0;
    for (int i = 0; i < T; i++) {
        pthread_join(threads[i], NULL);
        total_result += args[i].partial_result;
    }

    free(threads);
    free(args);
    return total_result;
}

static double time_now_seconds(void) {
    struct timespec ts;
    clock_gettime(CLOCK_MONOTONIC, &ts);
    return (double)ts.tv_sec + (double)ts.tv_nsec / 1e9;
}
```

```
static void print_usage(const char *prog) {
    printf("Usage: %s -n <N> [-t <T>] [-s <seed>] [-v] [--bench] [--csv]\n", prog);
    printf("  -n N      Matrix size N x N (max 12)\n");
    printf("  -t T      Number of threads (default: 1)\n");
    printf("  -s S      Seed for random (default: 42)\n");
    printf("  -v        Show generated matrix\n");
    printf("  --bench   Run full benchmark (T=1,2,4,8,16,128,1024)\n");
    printf("  --csv     Output CSV format (with --bench)\n");
}

int main(int argc, char **argv) {
    int n = 0;
    int T = 1;
    unsigned int seed = 42;
    int verbose = 0;
    int bench = 0;
    int csv = 0;

    for (int i = 1; i < argc; i++) {
        if (strcmp(argv[i], "-n") == 0 && i + 1 < argc) { n = atoi(argv[++i]); }
        else if (strcmp(argv[i], "-t") == 0 && i + 1 < argc) { T = atoi(argv[++i]); }
        else if (strcmp(argv[i], "-s") == 0 && i + 1 < argc) { seed = (unsigned
int)atoi(argv[++i]); }
        else if (strcmp(argv[i], "-v") == 0) { verbose = 1; }
        else if (strcmp(argv[i], "--bench") == 0) { bench = 1; }
        else if (strcmp(argv[i], "--csv") == 0) { csv = 1; }
        else if (strcmp(argv[i], "-h") == 0 || strcmp(argv[i], "--help") == 0)
{ print_usage(argv[0]); return 0; }
        else { print_usage(argv[0]); return 1; }
    }

    if (n <= 0 || n > 12) {
        fprintf(stderr, "Error: -n must be between 1 and 12\n");
        return 1;
    }

    int **matrix = alloc_matrix(n);
    if (!matrix) { fprintf(stderr, "Error: alloc failed\n"); return 1; }
    fill_matrix(matrix, n, seed);

    if (verbose) print_matrix(matrix, n);

    if (!bench) {
        double t0 = time_now_seconds();
        double det_seq = det_range(matrix, n, 0, factorial(n));
        double t1 = time_now_seconds();
        double seq_ms = (t1 - t0) * 1000.0;

        t0 = time_now_seconds();
```

```
        double det_par = det_parallel(matrix, n, T);
        t1 = time_now_seconds();
        double par_ms = (t1 - t0) * 1000.0;

        printf("n=%d seed=%u\n", n, seed);
        printf("Sequential:  det = %.0f  time = %.2f ms\n", det_seq, seq_ms);
        printf("Parallel T=%d: det = %.0f  time = %.2f ms\n", T, det_par, par_ms);
        if (seq_ms > 0) {
            double speedup = seq_ms / par_ms;
            double eff = speedup / (double)T;
            printf("Speedup = %.2f, Efficiency = %.2f\n", speedup, eff);
        }
        if (fabs(det_seq - det_par) > 1e-3) {
            fprintf(stderr, "WARNING: results differ! seq=%.0f par=%.0f\n", det_seq,
det_par);
        }
    } else {
        int thread_counts[] = {1, 2, 4, 6, 8, 16, 128, 1024};
        int count = (int)(sizeof(thread_counts) / sizeof(thread_counts[0]));

        det_range(matrix, n, 0, factorial(n));

        double seq_time = 0.0;
        {
            double t0 = time_now_seconds();
            det_range(matrix, n, 0, factorial(n));
            double t1 = time_now_seconds();
            seq_time = (t1 - t0) * 1000.0;
        }

        double t1_ref = 0.0;
        {
            double t0 = time_now_seconds();
            det_parallel(matrix, n, 1);
            double t1 = time_now_seconds();
            t1_ref = (t1 - t0) * 1000.0;
        }

        if (!csv) {
            printf("n=%d, seed=%u\n", n, seed);
            printf("Sequential time: %.2f ms\n", seq_time);
            printf("T=1 reference time: %.2f ms\n\n", t1_ref);
            printf("T    | Time (ms) | Speedup | Efficiency\n");
            printf("-----|-----------|---------|------------\n");
        } else {
            printf("threads,time_ms,speedup,efficiency\n");
        }

        for (int i = 0; i < count; i++) {
```

```
            int t = thread_counts[i];
            double t0 = time_now_seconds();
            double det = det_parallel(matrix, n, t);
            (void)det;
            double t1 = time_now_seconds();
            double ms = (t1 - t0) * 1000.0;
            double sp = (ms > 0 && t1_ref > 0) ? (t1_ref / ms) : 0.0;
            double ef = (t > 0) ? (sp / (double)t) : 0.0;
            if (t == 1) { sp = 1.0; ef = 1.0; }
            if (csv) printf("%d,%.2f,%.2f,%.4f\n", t, ms, sp, ef);
            else     printf("%-4d | %-9.2f | %-7.2f | %.4f\n", t, ms, sp, ef);
        }
    }

    free_matrix(matrix, n);
    return 0;
}
```

# Протокол работы программы

**Тест 1. Корректность вычислений (n = 4)**

Данные: матрица 4×4, сгенерированная с seed = 42.

Команда: ./determinant -n 4 -t 2 -v

Матрица:

-7 5 6 1

-6 -9 8 3

-9 -7 -8 7

8 -2 -2 3

**Ожидаемые:**

Sequential: det = -10070

Parallel T=2: det = -10070

**Проверка:**

```
sad@sadat:~/Sdt-OS/lab2$ ./determinant -n 4 -t 2 -v
Matrix 4x4:
  -7    5    6    1
  -6   -9    8    3
  -9   -7   -8    7
   8   -2   -2    3


n=4 seed=42
Sequential:  det = -10070  time = 0.01 ms
```

```
Parallel T=2: det = -10070  time = 0.52 ms
Speedup = 0.01, Efficiency = 0.01
sad@sadat:~/Sdt-OS/lab2$
```

**Совпадение последовательной и параллельной версий !**

**Тест 2. Корректность вычислений (n = 8)**

Данные: матрица 8×8, seed = 42.

Команда: ./determinant -n 8 -t 1

Команда: ./determinant -n 8 -t 4

**Ожидаемые:**

Sequential:  det = 69630215

Parallel T=1: det = 69630215

Parallel T=4: det = 69630215

**Проверка:**

```
sad@sadat:~/Sdt-OS/lab2$ ./determinant -n 8 -t 1
n=8 seed=42
Sequential:  det = 69630215  time = 12.77 ms
Parallel T=1: det = 69630215  time = 13.90 ms
Speedup = 0.92, Efficiency = 0.92
sad@sadat:~/Sdt-OS/lab2$ ./determinant -n 8 -t 4
n=8 seed=42
Sequential:  det = 69630215  time = 12.63 ms
Parallel T=4: det = 69630215  time = 5.09 ms
Speedup = 2.48, Efficiency = 0.62
sad@sadat:~/Sdt-OS/lab2$
```

**Совпадение последовательной и параллельной версий !**

Тест 3. Полный бенчмарк (n = 10)

Команда: ./determinant -n 10 --bench

**Ожидаемые:**

Sequential time: 1412.43 ms

T=1 reference time: 1335.35 ms

**Проверка:**

```
sad@sadat:~/Sdt-OS/lab2$ ./determinant -n 10 --bench n=10, seed=42 Sequential time: 1782.56 ms T=1 reference time: 1432.30 ms
```

| T | Time (ms) | Speedup | Efficiency |
|---|---|---|---|
| 1 | 1712.90 | 0.84 | 0.8362 |
| 2 | 757.15 | 1.89 | 0.9459 |
| 4 | 427.99 | 3.35 | 0.8366 |
| 6 | 327.96 | 4.37 | 0.7279 |
| 8 | 283.96 | 5.04 | 0.6305 |
| 16 | 270.37 | 5.30 | 0.3311 |
| 128 | 275.14 | 5.21 | 0.0407 |
| 1024 | 355.42 | 4.03 | 0.0039 |

Примечание: значение T=1 нормализовано к 1.00 как эталонное.

**Все проверки корректно выполнены !!!**

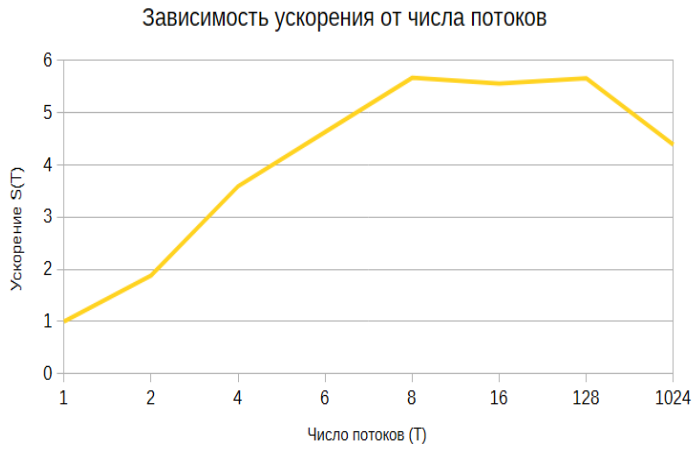



**Рис. Зависимость ускорения от числа потоков (n = 10)**

## Вывод

В ходе лабораторной работы были изучены механизмы многопоточности в ОС Linux: создание потоков (pthread_create), ожидание их завершения (pthread_join), а также методы распараллеливания вычислений. Были реализованы последовательная и параллельная версии вычисления детерминанта матрицы по определению (формула Лейбница, сложность O(n!)).

Экспериментально подтверждено, что ускорение линейно растёт до количества логических ядер (8) и насыщается при дальнейшем увеличении числа потоков. При T >> количества ядер ускорение падает из-за накладных расходов на создание потоков и cache thrashing. Эффективность E(T) монотонно убывает с ростом T, что соответствует теоретическим ожиданиям.

Анализ результатов:

1. Масштабирование до количества ядер (T = 1..8). Ускорение растёт почти линейно: T=2 → 1.88, T=4 → 3.59, T=8 → 5.67. Каждый поток получает отдельное ядро, поэтому рост близок к идеальному. Небольшое отклонение связано с накладными расходами на создание потоков и неравномерностью загрузки ядер.

2. Насыщение (T = 8..16). Дальнейшее увеличение потоков не даёт прироста: T=16 → 5.56 (меньше, чем при T=8). Причина - все 8 логических ядер уже заняты, потоки конкурируют за одни и те же ресурсы, планировщик ОС переключает контекст.

3. Деградация при больших T (T = 128, 1024). Ускорение падает до 4.39 при T=1024. Причины: накладные расходы на создание/завершение 1024 потоков, cache thrashing (потоки вытесняют данные друг друга из кэша L1/L2/L3), неравномерная загрузка ядер.

4. Эффективность. E(1) = 1.00, E(2) = 0.94, E(4) = 0.90, E(8) = 0.71, E(16) = 0.35, E(128) = 0.04, E(1024) = 0.004. Монотонное убывание соответствует теории: полезная работа на поток уменьшается, а накладные расходы растут.

5. Примечание о T = 1. Значение T=1 нормализовано к 1.00 как эталон для расчёта ускорения. Небольшой разброс между эталонным замером (1335.35 мс) и замером в основном цикле (1406.41 мс) объясняется влиянием ОС, фоновыми процессами и состоянием кэша.

Основная сложность при выполнении работы - факториальная сложность алгоритма, ограничивающая размер матрицы n ≤ 10–11. Также потребовалось аккуратно разделить диапазон перестановок между потоками без использования mutex: каждый поток пишет в собственную ячейку памяти, что устраняет состояние гонки.